

NUS Orbital 2026



Milestone 01

[Poster](#) [Video](#) [App](#) [Logs](#)

Atlas (Team ID:)
Wang Qichen (A0337219B)
Feng Yi (A0322277E)

Proposed Level of Achievement:
Artemis

Contents

Deployment	4
Artemis M1 Resubmission Coverage	4
Motivation	5
Our Aim	5
User Stories	5
Overall Architecture Diagram	7
Core Features	8
Feature Ideation	10
Repository and Project Foundation	12
Project Structure and Version Control	12
Project-Level Software Engineering	12
GitHub Project Management	13
Engineering Principles	13
Risk Management	14
Authentication and Protected Access	15
Email, Demo, and Google Sign-In	15
Protected Routes	15
Campus Map Rendering	17
MapLibre and OSM Tiles	17
Route Tracing	17
Daily Assistant, Facilities, and Schedule	19
Facility Discovery	19
Schedule Management	19
Recommendation Cards	19
NUSMods Sync and Bus Context	20
NUSMods Sync	20

Bus and Transit Context	20
Mobile Frontend Experience	21
Responsive Layout	21
Standardised Software Diagrams	22
ER Diagram	22
Sequence Diagram	23
Activity Diagram	24
Automated Testing and QA	25
Automated Checks	25
Manual Test Matrix	26
CI/CD Pipeline	27
Continuous Integration Gates	27
Continuous Deployment Path	27
Video Walkthrough	29
Milestone 01 Progress	30
Planned Next Steps	30

Deployment

The deployed web application is available at: orbital-artemis-armap-nus.onrender.com

Artemis M1 Resubmission Coverage

This revised Milestone 01 report is prepared as an Artemis-level resubmission. It explicitly addresses the evaluation feedback by expanding the product scope, documenting project-level software engineering practices, recording automated testing and CI/CD plans, and linking the implementation to a video walkthrough.

S/N	Artemis Expectation	Where Addressed in This Report
1	20–23 user stories	The User Stories section now contains 20 user stories covering students, visitors, accessibility needs, campus transport, facilities, safety, and community map updates.
2	Ideation for at least 9 features	The Core Features and Feature Ideation sections document 10 product and engineering features with user value, implementation direction, and M1 status.
3	Software engineering at project level	The Project-Level Software Engineering section documents GitHub-based project management, branching, module boundaries, API contracts, deployment discipline, and risk management.
4	Artemis M1 report depth	The report is expanded beyond a basic prototype summary to cover architecture, feature ideation, testing, CI/CD, deployment, and planned iteration.
5	Automated testing documentation	The Automated Testing and QA section documents current automated checks, planned unit/API/UI tests, and the manual test matrix used for M1 validation.
6	CI/CD pipeline documentation	The CI/CD Pipeline section documents the intended GitHub Actions workflow, build gates, Docker deployment path, and Render production deployment.
7	Video walkthrough	The Video Walkthrough section describes the submitted walkthrough flow and maps it to the current app features.

Motivation

Navigating through the NUS campus can be difficult because complete geospatial and indoor facility data is not always publicly available or intuitive to use. Even after reaching the correct building, students may still need to locate lecture rooms, tutorial classrooms, study spaces, restrooms, lifts, printing services, and other facilities.

These navigation needs are closely tied to students' daily routines and schedules. Atlas is motivated by the need for a smarter, more context-aware campus assistant that supports both outdoor movement across NUS and indoor discovery within selected buildings.

Our Aim

Atlas aims to create a seamless campus assistant for NUS students. The project combines map-based navigation, facility discovery, schedule-aware guidance, and a daily assistant interface so that students can move through campus more confidently.

For Milestone 01, our goal is to establish a working full-stack prototype: authentication, a protected dashboard, campus data APIs, map rendering, location search, and deployment. Later milestones will extend this foundation toward richer AR-style navigation, indoor guidance, and smarter route planning.

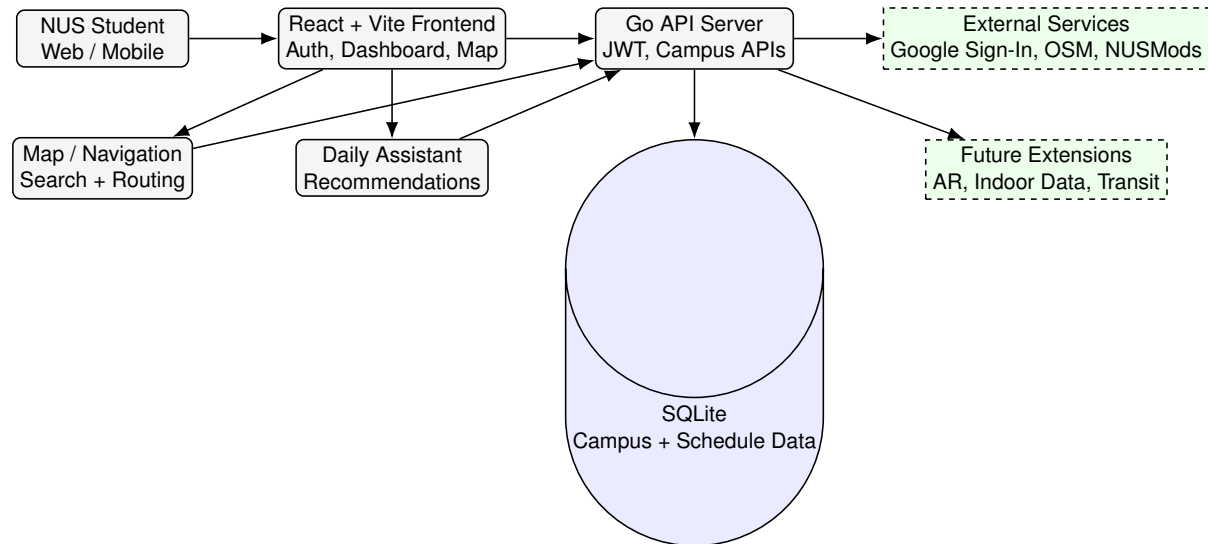
User Stories

1. As an NUS student, I can use the AR map to navigate from my current location to the correct building and classroom so that I can arrive at class on time.
2. As an NUS student unfamiliar with a campus building, I can locate indoor facilities such as restrooms, lifts, study spaces, and printing services so that I can navigate the building more easily.
3. As an NUS student, I can receive location-aware reminders and suggestions based on my schedule and current position so that I can manage my day more smoothly.
4. As a freshman unfamiliar with NUS, I can search for lecture theatres, tutorial rooms, and laboratories so that I can navigate the campus confidently.
5. As an exchange student visiting NUS for the first time, I can follow AR navigation instructions instead of static maps so that navigation feels more intuitive.
6. As a student rushing between consecutive classes, I can estimate travel time between buildings so that I know when to leave for class.
7. As a student studying late on campus, I can locate nearby study spaces and facilities through AR guidance so that I can find suitable places quickly.
8. As a wheelchair user, I can find wheelchair-friendly routes, accessible entrances, and lifts so that I can travel across campus more comfortably.
9. As a student unfamiliar with NUS bus routes, I can locate nearby shuttle bus stops and view available shuttle services so that I can travel more efficiently.

10. As a student attending events on campus, I can receive recommendations for nearby ongoing events and activities so that I can participate more conveniently.
11. As a student looking for a quiet place to study, I can receive recommendations for less crowded study areas and facilities so that I can study more effectively.
12. As a student entering a large building for the first time, I can receive indoor AR guidance to classrooms, offices, and facilities so that I do not need to rely on physical signboards.
13. As a student who frequently prints documents on campus, I can quickly locate nearby printing services so that I can save time.
14. As a student concerned about safety, I can locate emergency exits, first-aid points, and security offices within campus buildings so that I can respond quickly during emergencies.
15. As a student contributing to the campus community, I can add or update map information such as facilities, pathways, and corrections so that the map remains accurate and useful.
16. As a student with back-to-back classes, I can receive reminders on when to leave my current location so that I can arrive on time for my next lesson.
17. As a visitor attending an event at NUS, I can navigate to specific venues and facilities so that I can find locations without prior knowledge of the campus layout.
18. As a student using the system daily, I can save frequently visited locations so that I can access navigation more quickly.
19. As a student who wants to avoid crowded routes, I can view alternative paths and facility options so that I can choose a route that matches my comfort and time constraints.
20. As a campus administrator maintaining navigation data, I can review user-submitted map corrections before they are published so that the system remains accurate and trustworthy.

Overall Architecture Diagram

The architecture follows the proposal's modular design: a user-facing interface, an authentication layer, a campus assistant/dashboard layer, a navigation and map layer, backend APIs, persistent data, and external campus services.



Atlas overall architecture for Milestone 01 and planned Artemis extensions

Core Features

The feature list below is based on the GitHub issue tracker, current repository implementation, and the product direction described in the original proposal. For Artemis Milestone 01, Atlas is treated as more than a small demo: it is a full-stack foundation for campus navigation, schedule-aware assistance, and future AR/indoor extensions.

S/N	Feature	Completed At	Brief Explanation
Core Features			
1	Repository and project foundation	Milestone 01	Issue #1 established the repository, README skeleton, license, and basic development structure.
2	Authentication and protected access	Milestone 01	Issues #2 and #8 cover email/password login, Google Sign-In, JWT issuing, and protected dashboard routing.
3	Campus map rendering	Milestone 01	Issues #7 and #17 cover OpenStreetMap/MapLibre rendering, marker display, and fixing building polygon display.
4	Location search and route tracing	Milestone 01	Issue #12 adds a searchable map interface with suggestions, map centering, and route-oriented interaction.
5	Daily assistant and campus data UI	Milestone 01	Issue #11 connects schedule, recommendations, buildings, facilities, and sync status into a student-facing assistant dashboard.
6	Facility discovery and filtering	Milestone 01	The dashboard exposes seeded campus buildings and facilities with filters for building code and facility type.
7	Schedule management	Milestone 01	Users can view, create, and delete schedule items, allowing later recommendation logic to reason about upcoming locations.
8	NUSMods and campus data sync	Milestone 01	Backend sync status and manual sync endpoints prepare the app for external academic timetable and campus-data integration.
9	Campus bus and transit context	Milestone 01 / Extension	Backend bus modules and API routes support bus stops, routes, arrivals, active vehicles, alerts, and nearby transit context.
Supporting / Usability Features			
10	Mobile frontend and iOS shell	Milestone 01	Issue #10 tracks responsive layout and manual viewport checks; the Capacitor iOS shell loads the deployed HTTPS app for mobile demo purposes.

Feature Ideation

The ideation below records the user problem, proposed solution, and engineering direction for all 10 planned features. This makes the scope explicit and prevents the implementation from becoming a collection of unrelated screens.

S/N	Feature	User Problem / Ideation	M1 Engineering Direction
1	Repository and project foundation	The team needs a shared base for code, documentation, issue tracking, deployment, and mentor review.	Maintain a monorepo with separated frontend, backend, deployment, and report files; use GitHub issues as logs and link implementation work to milestone evidence.
2	Authentication and protected access	A campus assistant may eventually contain schedule and preference data, so users need a trusted entry point.	Implement email/password login, registration, demo mode, Google Sign-In verification, JWT issuing, and protected routes.
3	Campus map rendering	Students need an interactive spatial view of campus rather than static text instructions.	Use MapLibre with OpenStreetMap tiles, a map engine helper, markers, route layers, and modular map services.
4	Location search and route tracing	Users should not need to pan manually across campus to find places or preview a trip.	Provide search suggestions, selected-place centering, route line drawing, and a route-oriented UI that can later connect to richer pathfinding.
5	Daily assistant dashboard	Navigation decisions depend on schedule, next destination, facility availability, and sync state.	Build a dashboard combining schedule cards, recommendation cards, building context, facilities, and sync status.
6	Facility discovery and filtering	Indoor needs such as toilets, lifts, printers, and study areas are often discovered only after entering a building.	Seed building and facility data in SQLite, expose protected API routes, and filter results by building and facility type.
7	Schedule management	Schedule-aware navigation requires structured upcoming events rather than hard-coded demo text.	Provide API and UI support for reading, adding, and deleting schedule items, with location fields tied to campus building codes.
8	NUSMods and campus data sync	A real NUS assistant should eventually update from official academic data instead of relying only on static seed data.	Add NUSMods client configuration, sync status endpoints, manual sync trigger, and scheduler configuration for later automation.

9	Bus and transit context	Campus movement is multimodal: students choose between walking and shuttle buses depending on time and location.	Add backend bus modules and endpoints for stops, routes, arrivals, active buses, alerts, and nearby bus context, with demo fallback data when live keys are absent.
10	Mobile frontend and iOS shell	Campus navigation is primarily mobile, so desktop-only usability is insufficient.	Improve responsive layouts, test small viewport behavior manually, and provide a Capacitor iOS shell that loads the deployed app for mobile demonstration.

Repository and Project Foundation

This feature establishes the engineering foundation for Atlas. From the user's point of view, it makes the project accessible as a real deployed application rather than a static proposal: the repository contains reproducible setup instructions, a clear project structure, and a production deployment path.

Project Structure and Version Control

Atlas is organized into a frontend, backend, deployment configuration, and supporting documentation. This mirrors the modular design described in the proposal and keeps responsibility boundaries clear. The frontend owns user interaction, the backend owns authentication and campus data APIs, and the deployment files define how the system runs in production.

```
Orbital_Artemis_ArMap_Nus/  
|-- frontend/ React + Vite frontend and mobile app shell  
| |-- src/ Auth, dashboard, campus map, and UI code  
| '-- ios/ Capacitor iOS project  
|-- backend/ Go API server  
| |-- cmd/server/ Server entry point  
| '-- internal/atlas/ Auth, campus data, routing, and API logic  
|-- deploy/ Production deployment configuration  
|-- poster_work/ Poster and presentation assets  
|-- Dockerfile Container build definition  
|-- docker-compose.yml Local multi-service run configuration  
'-- readme.tex Milestone report source
```

Project Structure

The repository uses GitHub issues and pull requests as progress logs. Issue #1 initialized the repository and documentation baseline, while later issues track authentication, map rendering, search, dashboard integration, and mobile usability. This supports the proposal's branch-based workflow: tasks are decomposed into issues, implemented on feature branches, reviewed through pull requests, and eventually integrated into `main`.

The main trade-off of this structure is that it creates more files and coordination overhead than a single-folder prototype. However, the benefit is significant for an Artemis-level project: each module can evolve independently, deployment can be reproduced, and later AR/navigation extensions can be added without rewriting the whole codebase.

Project-Level Software Engineering

Atlas applies software engineering practices at the project level, not only inside individual files. The goal is to make the project trackable, reviewable, testable, and deployable throughout Orbital.

GitHub Project Management

GitHub issues are used as the team progress log. Each issue represents a concrete feature, defect, or engineering task, such as authentication, protected routing, map rendering, mobile layout, dashboard integration, and map polygon fixes. The public issue tracker functions as both a task board and an audit trail for mentor review.

The intended workflow is:

1. Break milestone requirements into GitHub issues with clear acceptance criteria.
2. Assign work to a team member and connect it to the relevant feature area.
3. Implement changes on a focused branch instead of mixing unrelated work.
4. Review the change through a pull request before merging to `main`.
5. Use the issue and pull request history as progress logs for milestone submission.

This workflow is intentionally heavier than a quick prototype workflow. It is justified because Artemis projects are expected to demonstrate consistent engineering discipline and maintainability, not only visible screens.

Engineering Principles

Principle	How Atlas Applies It
Separation of concerns	Frontend pages and UI components are separated from backend API handlers, authentication logic, storage logic, and deployment configuration.
API-first integration	Frontend features call documented backend endpoints such as <code>/api/buildings</code> , <code>/api/facilities</code> , <code>/api/schedule</code> , <code>/api/recommendations</code> , and bus endpoints.
Progressive delivery	The team implements a reliable 2D map and schedule-aware dashboard before adding higher-risk AR and indoor navigation layers.
Security by default	Protected pages require authentication, backend data routes are guarded with JWT middleware, and Google ID token verification happens server-side.
Reproducibility	Local setup, environment variables, Docker build, Docker Compose deployment, and production Caddy configuration are documented in the repository.
Graceful degradation	Demo credentials, seeded data, and bus demo fallback data allow reviewers to evaluate the project even without private external API keys.

Risk Management

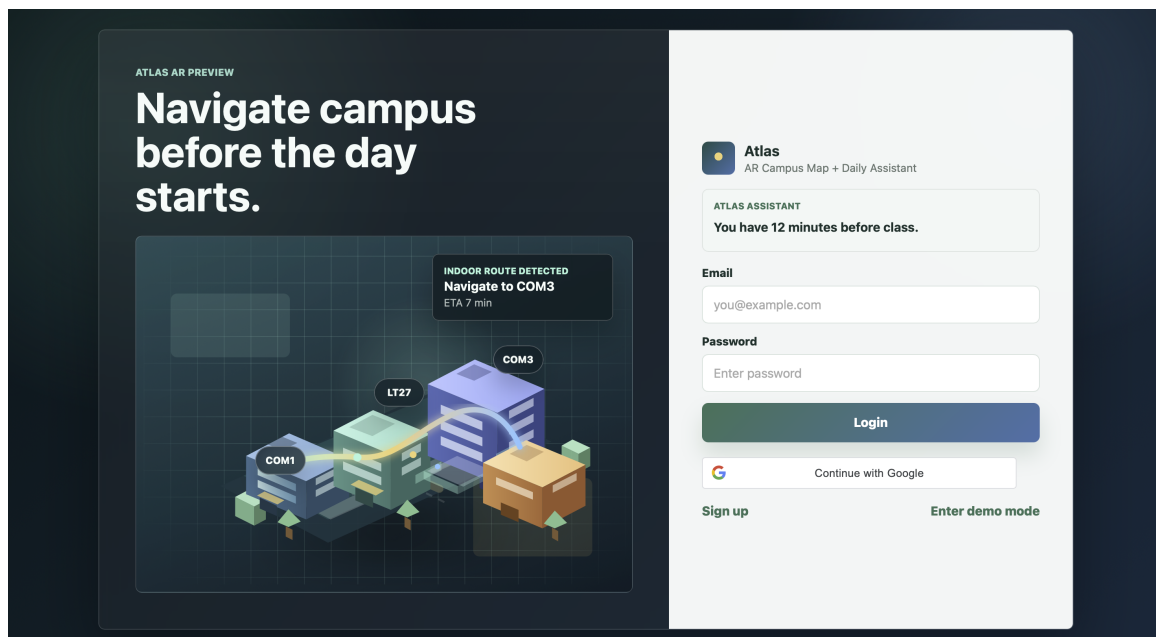
Risk	Impact	Mitigation
Incomplete campus/indoor data	The assistant may not cover every building or facility at M1.	Use seeded demo data for core flows, design APIs around building/facility models, and expand coverage in later milestones.
External API reliability	NUSMods, Google verification, map tiles, or bus APIs may fail or require credentials.	Keep configuration in environment variables and provide demo/fallback behavior where possible.
AR complexity	Full AR navigation may exceed M1 implementation capacity.	Build a stable map, routing, and data foundation first, then integrate AR overlays after core navigation behavior is reliable.
Mobile usability defects	Navigation apps are frequently used on phones, where cramped layouts can break flows.	Track responsive layout as a feature, include mobile screenshots, and test common auth/dashboard/map flows on small viewports.

Authentication and Protected Access

Authentication allows students to enter Atlas through email/password login, registration, demo mode, or Google Sign-In. Once authenticated, users can access protected pages such as the dashboard and map.

Email, Demo, and Google Sign-In

The login system supports multiple entry paths. Email/password login provides a traditional account flow, demo mode allows quick testing without setup, and Google Sign-In supports a more realistic student login experience.



Authentication Screen

On the backend, Atlas verifies credentials and issues a JWT. For Google Sign-In, the frontend obtains a Google ID token and sends it to the backend, where it is verified against the configured Google client ID. If verification succeeds, the backend returns the same application JWT format used by email login. This keeps the rest of the application independent of the authentication method.

From a software engineering perspective, this follows separation of concerns. The frontend handles user interaction and token storage, while the backend performs trust-sensitive verification and token signing. This is safer than trusting the frontend alone, because the backend remains the authority for protected access. The trade-off is that local development requires OAuth configuration and network access to Google's verification service, but the design is closer to a production-ready authentication system.

Protected Routes

Protected routes guard the dashboard and map pages. If a user is not authenticated, the frontend redirects them to the login screen. API requests also attach the JWT in the `Authorization` header, allowing the backend to reject unauthenticated access consistently.

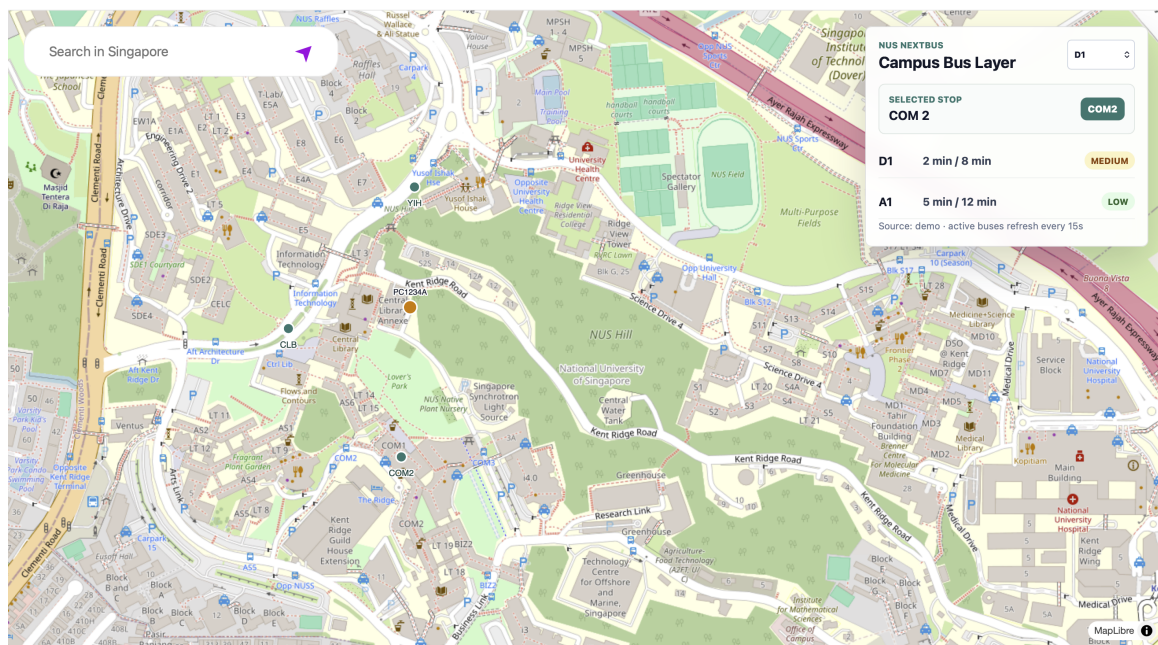
This pattern reduces duplicated access checks. Instead of manually checking login state inside every page component, protected routing creates a reusable access boundary. It also makes later feature additions easier, because new protected pages can reuse the same wrapper.

Campus Map Rendering

The map feature gives users an interactive visual interface for campus navigation. For Milestone 01, Atlas renders a MapLibre/OpenStreetMap-based campus map centered around NUS and prepares the map surface for future AR and indoor navigation work.

MapLibre and OSM Tiles

The frontend uses MapLibre to render map tiles and manage markers, routes, and future geospatial overlays. This gives the project an open-source map engine that can be extended with custom campus data instead of relying only on a closed map UI.



Campus Map Screen

The map logic is separated into a map initialization module, a MapEngine helper, routing services, and UI components. This modular approach prevents the main map page from becoming a single large component. It also makes it easier to add bus layers, indoor facility overlays, or AR-style markers later.

The bug tracked in issue #17 showed why this separation matters. Rendering geospatial polygons and map layers can fail due to data format mismatches, layer ordering, or source initialization timing. By separating map engine actions from UI state, Atlas can debug and update rendering logic without changing authentication or dashboard code.

Route Tracing

Route tracing allows selected start and end points to be drawn on the map. In Milestone 01, this is implemented as a route line on the MapLibre layer. It is an early prototype of the proposal's broader navigation goal.

The engineering trade-off is that route tracing is currently web-map based rather than full AR. This is intentional for Milestone 01: a reliable map and routing foundation reduces risk before

adding heavier AR components in later milestones.

Daily Assistant, Facilities, and Schedule

The daily assistant dashboard connects campus navigation to student routines. Instead of treating the map as a standalone feature, Atlas combines schedule items, next-location context, building data, facility filters, and recommendation cards into one student-facing workflow.

Facility Discovery

Facility discovery focuses on practical indoor and near-indoor needs: study areas, lifts, restrooms, and printing locations. In Milestone 01, the backend seeds representative NUS building and facility data in SQLite and exposes it through protected API endpoints.

Endpoint / Data	Purpose
GET /api/buildings	Returns campus buildings with code, name, description, coordinates, floor count, and indoor support status.
GET /api/facilities	Returns facilities and supports filters such as <code>building=COM1</code> and <code>type=study_space</code> .
Seeded demo data	Allows evaluators to test building and facility discovery without requiring private production data.

The main design decision is to model facilities separately from buildings. This allows Atlas to add many facility records under one building and later attach richer indoor information such as floor plans, accessibility paths, or AR markers.

Schedule Management

Schedule management is included because a campus assistant becomes more useful when it understands where the student needs to go next. The current prototype supports viewing, creating, and deleting schedule items. Each item can include a location code, allowing recommendation cards to connect time-based context with campus data.

This feature also prepares the app for future NUSMods integration. Rather than hard-coding a static timetable into the frontend, the backend exposes schedule APIs and stores schedule data in SQLite. This gives the team a stable place to add timetable sync, conflict warnings, and route preparation in later milestones.

Recommendation Cards

Recommendation cards summarize useful actions such as where the next class is, which building context matters now, or which facilities are likely relevant. In M1, the recommendations are intentionally simple and explainable. This helps the team test the user workflow before adding more advanced ranking or personalization.

NUSMods Sync and Bus Context

Atlas includes early integration points for external campus systems. These are important Artemis extensions because they move the project beyond a static campus map and toward a live assistant.

NUSMods Sync

The backend contains configuration for academic year, sync interval, HTTP timeout, and NUSMods integration. The dashboard exposes sync status and a manual sync trigger so users can see whether external data integration is healthy.

Endpoint / Setting	Purpose
GET /api/sync/status	Shows the latest sync state for campus or timetable data.
POST /api/sync/run	Allows a manual sync trigger during development and demonstration.
NUSMODS_ACAD_YEAR	Selects the academic year used by the NUSMods client.
SYNC_INTERVAL	Controls the scheduler interval for future automatic syncing.

Bus and Transit Context

Campus movement often involves a choice between walking, waiting for a shuttle, or combining both. Atlas therefore includes bus context as an Artemis extension feature. The backend supports endpoints for bus stops, routes, arrivals, active vehicles, alerts, and context near a latitude/longitude point.

The current design keeps bus data server-side. This avoids exposing private API keys in the browser and allows the backend to cache, normalize, or fall back to demo data. For M1, this establishes the integration boundary even if later milestones refine the frontend presentation and multimodal routing logic.

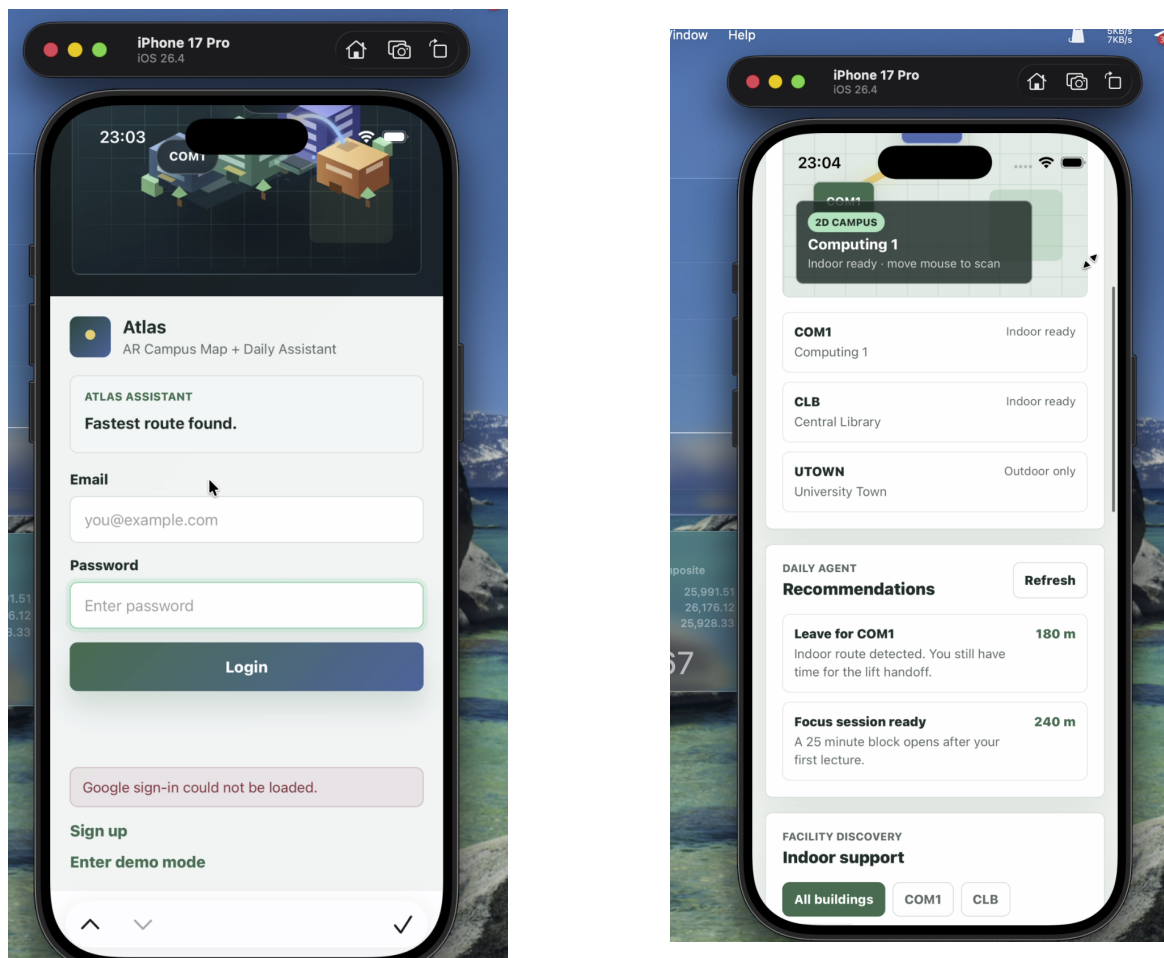
Bus API	Use Case
GET /api/bus/stops	List campus shuttle stops.
GET /api/bus/routes	List available NUS shuttle routes.
GET /api/bus/arrivals?stop=COM2	Show estimated arrivals for a selected stop.
GET /api/bus/active?route=D1	Show active vehicles for a route.
GET /api/bus/context?lat=...	Provide nearby transit context for assistant recommendations.

Mobile Frontend Experience

Because campus navigation is often used while students are moving, mobile usability is a supporting but important feature. Issue #10 tracks responsive layout and manual viewport checks for authentication, dashboard, map, schedule, and recommendations.

Responsive Layout

The frontend is designed to remain readable on common mobile widths. Cards, buttons, filters, and map panels are arranged to avoid horizontal scrolling and text overlap.



Mobile App Screens

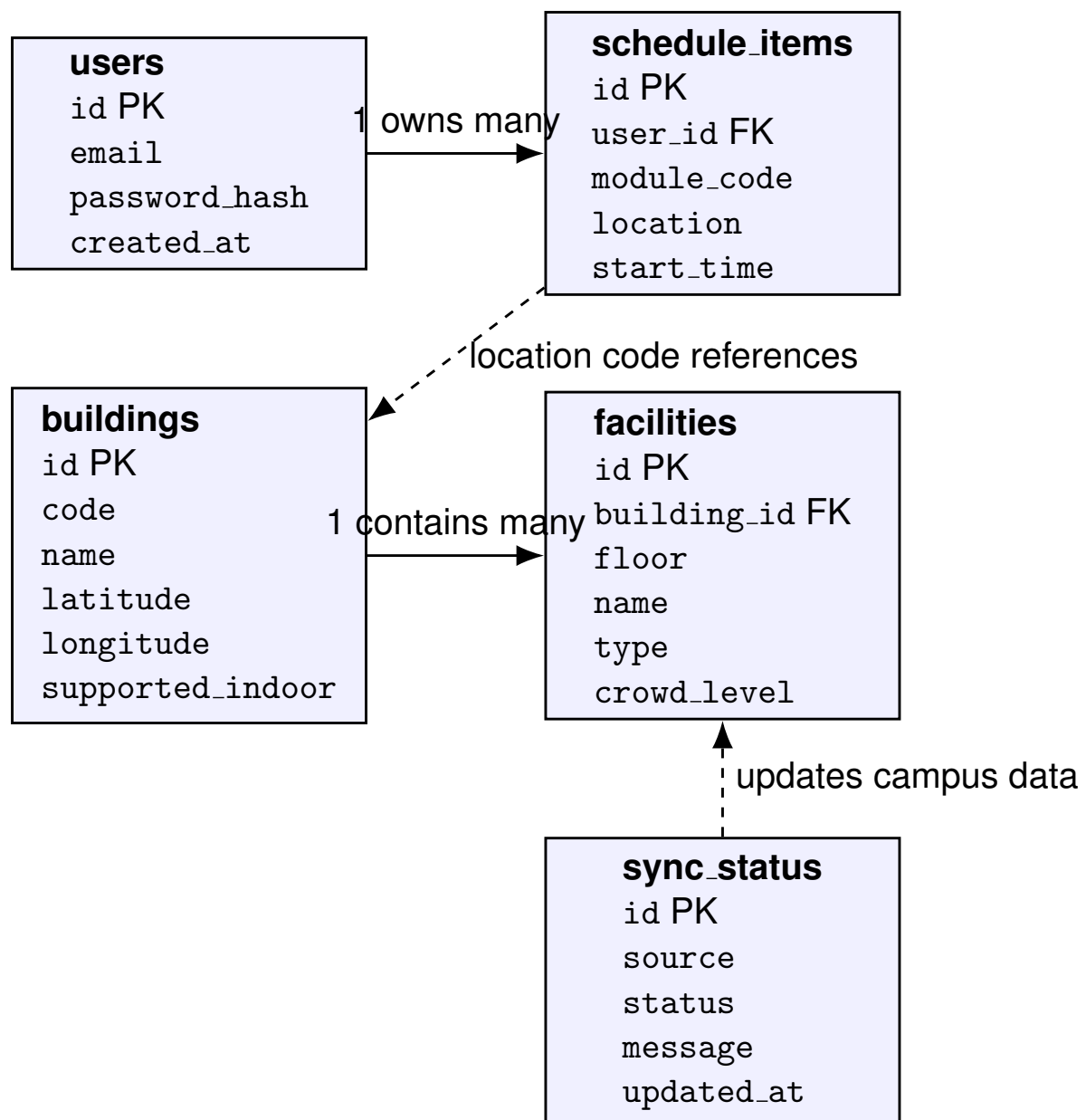
The main engineering principle here is progressive enhancement: the application should remain usable on desktop while also adapting to mobile. The trade-off is extra CSS and layout testing, but this matters for a campus assistant because students are more likely to use it on phones than on large desktop screens.

Standardised Software Diagrams

In addition to the high-level architecture diagram, the following standardised diagrams describe different aspects of Atlas: database relationships, authentication interaction order, and the map/search user journey.

ER Diagram

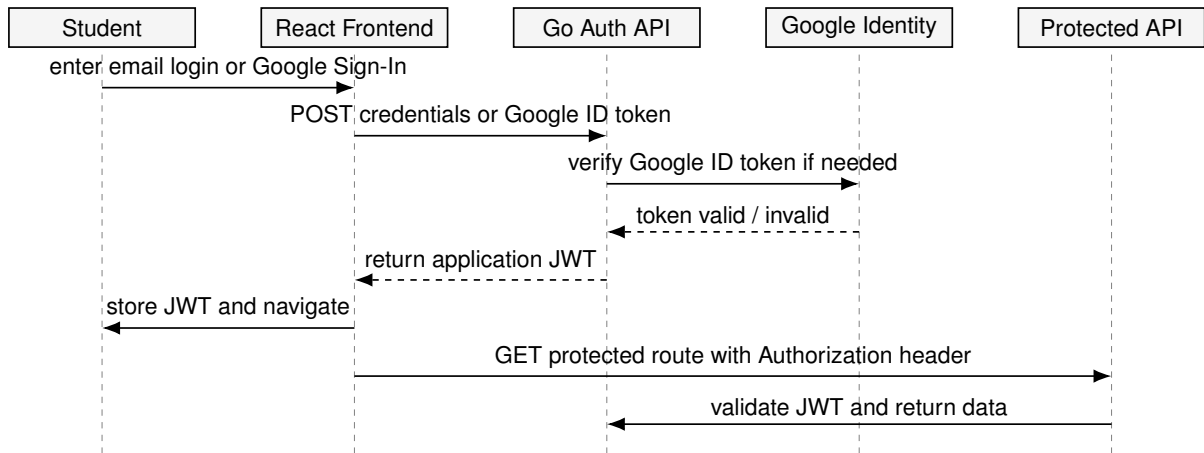
The ER diagram models the core persistent data needed for Milestone 01 and later Artemis extensions. Users own schedule items, buildings contain facilities, and sync status records document external data updates.



ER diagram for users, buildings, facilities, schedule items, and sync status

Sequence Diagram

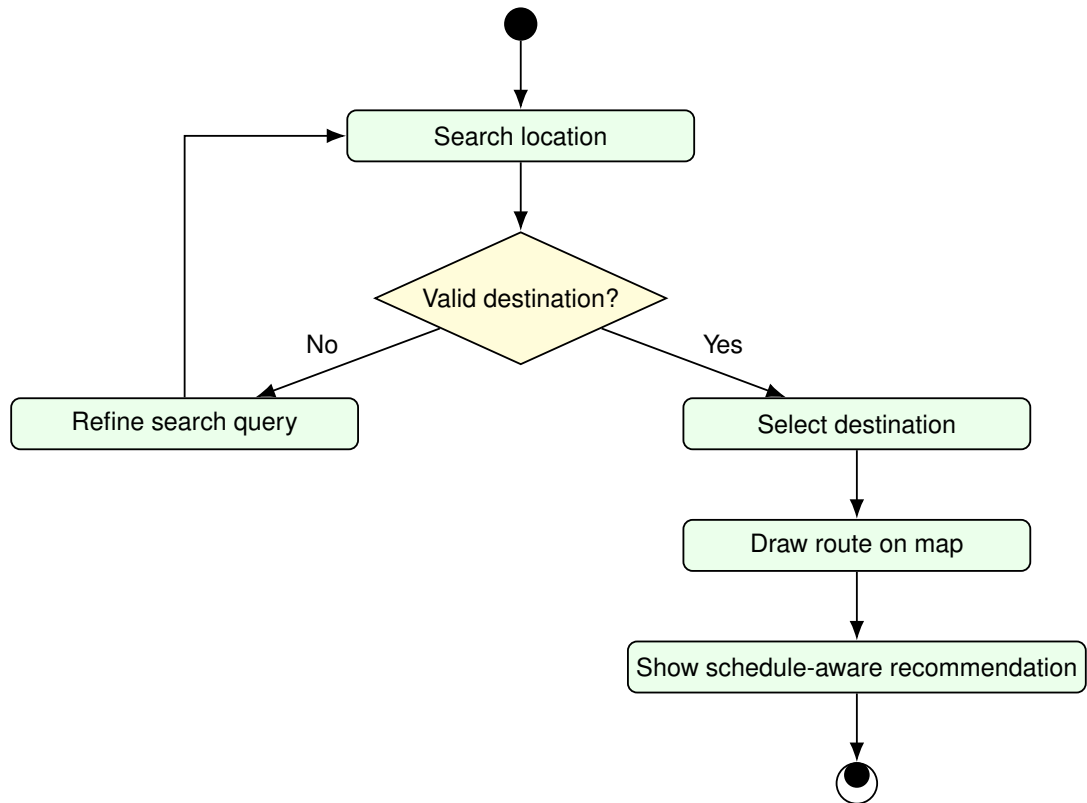
The sequence diagram shows the login, Google Sign-In, JWT issuing, and protected route access flow. The frontend may send either email/password credentials or a Google ID token, but the backend remains the authority that verifies identity and issues the application JWT.



Sequence diagram for login, Google Sign-In, JWT issuing, and protected route access

Activity Diagram

The activity diagram describes the user journey for map search and route-oriented recommendations. The flow starts from a location search, moves through destination selection and route rendering, and ends with contextual recommendations.



Activity diagram for search location, destination selection, route drawing, and recommendation display

Automated Testing and QA

Testing for Milestone 01 is split into automated checks and manual usability checks. Automated checks are used for repeatable engineering confidence, while manual checks are used where the current prototype depends on browser behavior, map rendering, OAuth, and mobile layout.

Automated Checks

Check	Command	What It Validates
Backend package tests	<code>cd backend && go test ./...</code>	Compiles backend packages and runs Go test files when present. This protects API handlers, store logic, auth logic, scheduler logic, and integration helpers from compile-time regressions.
Frontend production build	<code>cd frontend && npm run build</code>	Verifies that React, Vite, routing, TypeScript/JSX modules, and imported assets can produce a production bundle.
iOS shell sync build	<code>cd frontend && npm run ios:sync</code>	Runs the frontend build and synchronizes it with Capacitor iOS configuration for mobile demo validation.
Docker production build	<code>docker compose up -d --build</code>	Validates that the frontend and backend can be built into the production container and served through the configured deployment stack.

At this milestone, the strongest automated coverage is build-level and package-level validation. The next planned improvement is to add explicit unit tests for backend store/API behavior and frontend component tests for authentication, dashboard loading, schedule actions, and map UI state.

Manual Test Matrix

Flow Tested	Method	Expected Result
Demo login	Use seeded demo account test1@gmail.com / cp2106.	User reaches the protected dashboard and receives an application JWT.
Registration	Register a new account through the auth screen.	New credentials are accepted and can be used to access protected pages.
Google Sign-In	Sign in with a configured Google OAuth client.	Backend verifies the ID token and returns the same app JWT format.
Dashboard loading	Open protected dashboard after login.	Health, buildings, schedule, recommendations, facilities, and sync status appear.
Facility filtering	Change building and type filters.	Facility list updates without breaking dashboard layout.
Schedule actions	Add and delete schedule items.	Schedule list updates and recommendation context remains consistent.
Campus map	Open map page, search location, and draw route.	Map loads, marker/route interactions work, and the UI remains usable.
Mobile viewport	Test auth, dashboard, map, and mobile screenshots at narrow widths.	Text, cards, buttons, and map panels remain readable without horizontal scrolling.
Production app	Visit the Render deployment URL.	The deployed app loads over HTTPS and supports the same demonstration flow.

CI/CD Pipeline

The CI/CD pipeline is designed around GitHub as the source of truth and Render/Docker as the deployment path. Its purpose is to prevent unverified changes from reaching production and to make deployment reproducible.

Continuous Integration Gates

For pull requests and pushes to `main`, the intended GitHub Actions workflow runs the following jobs:

1. Checkout repository.
2. Install frontend dependencies with `npm ci`.
3. Build frontend with `npm run build`.
4. Install backend dependencies through Go modules.
5. Run backend checks with `go test ./...`
6. Build the production Docker image.

The corresponding workflow structure is:

```
name: Atlas CI
on: [push, pull_request]
jobs:
  frontend-build: npm ci + npm run build
  backend-test: go test ./...
  docker-build: docker build .
```

CI workflow gates for pull requests and main branch updates

Continuous Deployment Path

Atlas currently uses a production deployment path built from the repository's `Dockerfile`, `docker-compose.yml`, Caddy configuration, and Render-hosted application URL. The deployment flow is:

1. Merge reviewed changes into `main`.
2. Build the frontend with Vite.
3. Build the Go backend binary.
4. Package the production app through the multi-stage `Dockerfile`.
5. Serve the app over HTTPS through the configured deployment host.
6. Validate the deployed app using the manual test matrix and video walkthrough flow.

Environment variables such as `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `DB_PATH`, `ALLOWED_ORIGIN`, and bus API credentials are kept outside source code. This keeps deployment configurable while avoiding accidental exposure of private credentials.

Video Walkthrough

The submitted video is linked on the cover page and demonstrates the current usability of Atlas rather than only describing planned features. The walkthrough is structured to show a reviewer the app from first contact to core navigation workflows.

Step	Walkthrough Segment	Feature Evidence
1	App opening and deployment URL	Shows that the project is deployed and accessible through the submitted app link.
2	Authentication screen	Demonstrates email/demo login and explains Google Sign-In support.
3	Protected dashboard	Shows that unauthenticated access is separated from student-facing app content.
4	Schedule and recommendations	Demonstrates schedule-aware assistant behavior and next-location context.
5	Building and facility discovery	Shows campus data, filters, and facility cards.
6	Map screen and route interaction	Demonstrates map rendering, search-oriented interaction, and route tracing.
7	Mobile usability screenshots / iOS shell	Shows that the interface has been considered for phone use, not only desktop.
8	Engineering overview	Briefly points to repository structure, GitHub logs, testing commands, and deployment setup.

Milestone 01 Progress

Milestone 01 focused on ideation, planning, initial prototyping, and establishing a deployed technical foundation. The current implementation provides a working React frontend, Go backend, SQLite data layer, authentication flow, dashboard, map page, campus data APIs, and Render deployment.

The implementation also validates the proposed modular system design. Frontend pages, backend handlers, authentication logic, campus data storage, and map features are separated into maintainable modules, making it easier to extend the project in later milestones.

Planned Next Steps

- Expand the navigation layer toward richer indoor guidance and AR-style interaction.
- Improve campus data coverage for buildings, rooms, facilities, and navigation points.
- Continue the Sprint 3 work on transit, multimodal routing, and route visualization as extension features.
- Gather user feedback from NUS students and refine the assistant workflow.